

Evaluation Control

return

Description: Returns a value from a `function` expression.

Parameters:

- Value: The value to be returned.

Return: The input value.

Example:

```
(function (return 5)) ; Returns 5
```

function

Description: Evaluates the argument until it becomes `(return <result>)`, then reduces to `<result>`.

Parameters:

- Atom: The atom to be evaluated.

Return: The result of the atom's evaluation.

Example:

```
!(function (return (+ 1 2))) ; Returns 3 because (+ 1 2) evaluates to 3 which becomes (return 3)
```

eval

Description: Evaluates an atom, performing one step of reduction. This can be via equality rules or grounded functions.

Parameters:

- Atom: The atom to be evaluated.

Return: The result of the evaluation.

Example:

```
(= (double $x) (+ $x $x))  
!(eval (double 5)) ; Returns (+ 5 5)  
!(eval (+ 5 5)) ; Returns 10
```

evalc

Description: Evaluates an atom within a specific atomspace context.

Parameters:

- Atom: The atom to evaluate.
- Space: The atomspace in which to evaluate the atom.

Return: The result of the evaluation.

Example:

```
(= (double $x) (+ $x $x))  
!(evalc (double 5) &self) ; Returns (+ 5 5) self being the working space  
!(evalc (+ 5 5) &self) ; Returns 10
```

chain

Description: Evaluates an atom, binds the result to a variable, and then evaluates another atom containing the variable.

  [latest](#) ▼

Parameters:

- Atom: The atom to be evaluated initially.
- Variable: The variable to bind the result to.
- Template: The atom to evaluate after binding.

Return: The result of evaluating the template.

Example:

```
!(chain (+ 2 3) $x (* $x 2)) ; Evaluates (+ 2 3) to 5, binds it to $x, then evaluates (* $x 2), returning 10.
```

for-each-in-atom

Description: Applies a function passed as a second argument to each element of an atom passed as a first argument.

Parameters:

- Expression: The atom to apply the function to.
- Atom: The function to apply to each element of the atom.

Return: The result of applying the function to each element of the atom.

Example:

```
(= (print-each $x) (println! $x))
!(for-each-in-atom (1 3 5 62 2 5) print-each)
; Prints: 1 3 5 62 2 5 in separate lines
; Then it returns: ()
```